

Assignment: Image Classification using Convolutional Neural Networks (CNN)

In this assignment, you will build a **Convolutional Neural Network (CNN)** using the **Fashion-MNIST Dataset** to classify images of clothing into different categories.

The Fashion-MNIST dataset is an open-source dataset consisting of grayscale images of fashion products such as T-shirts, Shoes, Bags, Dresses, and Sneakers. It is widely used for learning computer vision and deep learning concepts.

Your goal is to build an end-to-end **CNN image classifier** using TensorFlow/Keras, train the model, evaluate its performance, and understand the role of convolutional layers in image classification.

Dataset Description

The assignment uses the **Fashion-MNIST Dataset**, which is available directly within TensorFlow/Keras.

Dataset Characteristics:

Attribute	Description
Number of Images	70,000
Training Images	60,000
Test Images	10,000
Image Size	28 × 28 pixels
Image Type	Grayscale
Number of Classes	10

The dataset contains images belonging to the following categories:

Label	Category
0	T-shirt / Top
1	Trouser
2	Pullover
3	Dress
4	Coat
5	Sandal
6	Shirt
7	Sneaker
8	Bag

Assignment Tasks

You must complete the following stages.

Part 1 — Data Preparation

Prepare the image dataset for training.

Tasks

1. Load the Fashion-MNIST dataset using TensorFlow/Keras.

```
from tensorflow.keras.datasets import fashion_mnist

(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
```

2. Display:
 - a. Number of training images
 - b. Number of testing images
 - c. Image dimensions
3. Normalize the image pixel values.
4. Reshape the images to make them compatible with CNN layers.
5. Display at least **five sample images** along with their class labels.

Part 2 — Building the CNN Model

Create a Convolutional Neural Network using TensorFlow/Keras.

Tasks

1. Import the required TensorFlow/Keras libraries.
2. Create a **Sequential CNN model**.
3. Include the following layers:
 - Convolution Layer (Conv2D)
 - Max Pooling Layer
 - Second Convolution Layer
 - Second Max Pooling Layer
 - Flatten Layer
 - Dense Hidden Layer
 - Output Layer
4. Use the following activation functions:

- ReLU for convolution and hidden layers
 - Softmax for the output layer
5. Compile the model using:
 - Optimizer: Adam
 - Loss Function: Sparse Categorical Crossentropy
 - Metric: Accuracy

Part 3 — Model Training

Train the CNN classifier.

Tasks

1. Train the model using the training dataset.
2. Select suitable values for:
 - a. Epochs
 - b. Batch Size
3. Store the training history.

Deliverables

- Training output
- Final training accuracy
- Number of epochs and batch size selected

Part 4 — Model Evaluation

Evaluate the trained CNN model.

Tasks

1. Evaluate the model on the testing dataset.
2. Generate predictions for the test images.
3. Display the predicted class labels for at least **10 test images**.
4. Evaluate the model using:
 - Test Accuracy
 - Confusion Matrix
 - Classification Report

Part 5 — Model Analysis

Answer the following questions:

1. Why are CNNs more suitable than Artificial Neural Networks (ANNs) for image classification?
2. What is the purpose of a Convolution layer?
3. Why is Max Pooling used in CNNs?
4. Why is the Flatten layer required before the Dense layers?
5. Why is the Softmax activation function used in the output layer?
6. How does the Adam optimizer help in training the CNN?